A Reusable Production System for Casual Puzzle Game Ad Creatives

Design, Implementation, and Production Practice

Dongyuan Li Yuwei Liu

Independent Researchers • Contact: Dongyuan Li • lzyzsere@gmail.com

Typesetting and figure note. This reading edition recovers the text of the final manuscript dated 20 September 2026, retaining its sections, tabulated data, original figure captions, and limitations. Original attachment bytes could not be fully exported. All five figures in this edition are explanatory schematics redrawn from the text, not original case screenshots or new experimental validation. This is a re-typeset reading edition of the authors' technical report, not an arXiv-published PDF.

Abstract

Casual puzzle-game advertisements require recurring changes to tile faces, materials, stacks, effects, and interaction pacing. In the team's generative editing workflow, preserving local edits across frames and maintaining plausible gameplay required repeated selection and rework. Meanwhile, professional source projects retained underused objects, animations, materials, and compositing relationships. This technical report presents Melo Game Creative Studio (melo-gcs) and a reusable production system integrating external agents, an element library, and professional authoring tools. Existing source projects can be edited directly or provide reusable elements for multimodal retrieval and assembly. When no source project is available, an agent can build an editable project from a reference video through the melo-gcs CLI. Projects separate gameplay recordings, presentation timing, and element references to support subsequent appearance, layout, action, and pacing edits. Chromium GPU rendering supports browser-based and headless video production; native After Effects or Blender refinement is used when browser output does not meet design requirements. Three cases illustrate source-project layout editing, save-and-reload checks at sampled frames, and variants made from original assets. Author-confirmed production records describe 18 groups of four variants: 72 videos were exported during a reported two-hour production stage, and 50 (69.4%) passed internal review and were deployed. Recorded model token charges were below US$1 per produced video, excluding other production costs. The report brings together implementation, cases, and production observations to document a practical route to advertisement variants through reuse of existing designs, domain-specific authoring, and browser rendering.

Keywords: casual puzzle-game advertising; source-project reuse; melo-gcs; external agents; gameplay authoring; headless rendering; Chromium

Contents

1 Motivation and System Contributions 2
2 Related Work and Technical Foundations 3
3 Production Tasks, Projects, and Deliverables 4
4 System Design and Division of Responsibilities 6
5 Browser Production and On-Demand DCC Refinement 9
6 Production Cases 11
7 Production Practice and Internal Acceptance 14
8 Design Experience and Scope 17
9 Conclusion 18
Materials Availability and Disclosure 18
References 19

1 Motivation and System Contributions

1.1 Two Challenges in Variant Production

Puzzle-game advertisements commonly develop an existing creative concept into multiple versions, changing tile faces, materials, stack arrangements, elimination effects, action sequences, or presentation timing. Producers need to explore new combinations while retaining useful parts of the existing design.

The team previously used video-generation and editing models to modify finished videos, drawing on reference-conditioned generation, video editing, and masked editing [2]. Although a tile face may occupy only a small part of the image, its identity and replacement mapping must remain consistent as it moves, becomes occluded, enters a tray, and is removed. A correct edit in one view must also persist throughout the sequence.

Gameplay adds a separate set of requirements: whether a tile is selectable, how a selection changes the tray or board, and which combinations may be eliminated must agree with the declared rules. Visual continuity and gameplay plausibility each affect whether an output is usable. These production experiences led us to move subsequent editing to addressable projects and objects, retaining content that did not need to change.

1.2 Underused Professional Source Projects

The team retained a substantial collection of After Effects compositions, Blender scenes, and legacy Cinema 4D assets. These preserve tile layers, objects, material references, animation, effects, and event arrangements that are flattened in a finished video, but this design work remained underused. Source projects represent an existing investment in design and provide directly accessible editing structures.

An agent invokes tools in the professional application to replace specified content in a project copy, retaining unaffected animation, tile bodies, and compositing relationships. Existing workflows cover tile-face, material, and effect edits; reference-guided layout tasks also check the resulting layers, occlusion, and interactions [3, 4].

As the number of tasks grew, assets distributed across projects needed a common means of discovery and reuse. Content suitable for independent use was extracted into an element library, where an agent can retrieve, select, and assemble it from textual requests or visual references. Content that is difficult to separate, or is better used as a whole, remains in complete projects.

1.3 From Project Editing to Domain-Specific Production

The layouts, timelines, and implementation choices of existing templates limit what can be changed. The team developed melo-gcs [1] to support new game states, different actions, and alternative elimination pacing through domain-specific interfaces. An external agent invokes existing operations through a command-line interface (CLI), continuing to edit the same project.

melo-gcs renders production videos through Chromium and the GPU. Headless execution supports tasks without a visible authoring interface. Designers review the output; when the browser's visual result does not meet the requirements, a dedicated import workflow creates a native After Effects or Blender project for further refinement. We use Skill to denote a reusable set of instructions that orchestrates tool calls for such a workflow. These digital content creation (DCC) applications support both reuse of existing design work and subsequent professional finishing.

When no source project is available, an external agent can use a reference video and the melo-gcs CLI to build a project. Once saved, subsequent edits operate on its objects, gameplay, and element references through the same browser production process.

1.4 Report Question and Three System Contributions

This report examines how underused professional design work, domain-specific software, and external agents can support continued editing and batch production as an alternative to direct video generation. The system retains existing designs at two levels: complete source projects and reusable elements. melo-gcs projects organise gameplay recordings, presentation timing, and element references, with domain operations for subsequent edits. Reference video extends intake to tasks without source projects. Three production cases and an observational record illustrate the system's use. The contributions concern three aspects of this production approach.

Reuse at project and element levels. The system supports targeted edits to complete professional projects and extraction of suitable assets and components for retrieval and combination in later tasks.

Agent-operable gameplay and presentation authoring. melo-gcs projects retain gameplay, presentation, and element references. Atomic CLI operations edit appearance, layout, actions, and pacing: the external agent plans and orchestrates, while melo-gcs executes domain operations.

Browser production with on-demand professional refinement. melo-gcs renders videos through Chromium GPU and headless execution, using DCC workflows when browser output falls short of design requirements. Hardware tests and a production record of 72 videos across 18 task groups illustrate this division of work.

2 Related Work and Technical Foundations

2.1 Video Generation and Project-Based Editing

VACE unifies reference-conditioned generation, video-to-video editing, and masked editing [2], operating directly on visual material. The video-input path in our system instead first establishes an editable melo-gcs project, which domain software renders and modifies. The source-project path uses existing objects and authoring relationships directly.

Each input type has preparation costs. Source projects may depend on particular assets and software environments, whereas video reconstruction requires identifying objects, actions, and appearance. This report examines reuse and editing after content has entered the system.

2.2 Creative Agents and Professional Software Control

BlenderAlchemy studies graphics editing with vision-language models [5]; BlenderGym evaluates code-driven graphics-editing systems [6]; and FilmAgent explores multi-agent filmmaking in virtual 3D spaces [7]. These works provide precedents for combining model decisions with creative tools.

The design question addressed here is how the result of a creative task becomes the editable input to the next one. Our system uses professional software control to retain existing design work and represents puzzle-game actions and presentation in melo-gcs projects. Reusing a validated gameplay recording makes appearance variation a separate operation from generating a new action sequence. This application-specific organisation complements the graphics-editing and filmmaking systems above; the present cases do not compare their task performance.

2.3 Browser Production Infrastructure

Chrome Headless supports browser execution without a visible interface, and Chromium documents GPU use under suitable configurations [8, 9]. Three.js provides a WebGL renderer and physically based material components [10, 11]. melo-gcs uses these foundations to organise geometry, materials, lighting, effects, and frame output for fixed or constrained viewpoints.

melo-gcs combines this existing infrastructure with project editing, gameplay authoring, and batch output. The system design centres on how these operations are organised into a production process.

3 Production Tasks, Projects, and Deliverables

3.1 Inputs and Editing Requirements

The system accepts professional source projects, reference videos, and saved melo-gcs projects. Textual requests and reference images specify target materials, tile-face themes, or stack outlines; new game states can also be created directly through melo-gcs configuration.

Each task identifies its input, requested changes, content to preserve, and delivery specifications. Tile-face replacement usually retains the original actions; retiming can preserve their order; changes to stacks or valid actions require the affected states and interactions to be reconsidered. A reference image supplies a visible layout, while hidden layers and complete gameplay require additional evidence.

3.2 melo-gcs Projects and Elements

A melo-gcs project is an editable, renderable creative object. It stores gameplay, presentation, and element references in JSON, forming a domain-specific language (DSL). We use "melo-gcs project" for the creative object and "melo-gcs project file" when discussing storage or transfer.

A project records how elements are combined and used. An element retains reusable content and its dependencies. Textures, scripts, effects, and components may be referenced by multiple compatible projects. Professional source projects retain their native editing model and can be used directly in their original applications.

In the public implementation, a gameplay recording (a Take) is associated with a game module, an initial-state identity, a seed, and ordered actions. Its semantic replay identity includes the action sequence but excludes pointer samples. Its frame replay identity additionally includes interactions, presentation timing, frame rate, and total frame count. A creative master carries game and ruleset versions, replay identities, layout and camera references, and output settings. A variant recipe selects appearance assets and permitted overrides; the compiler resolves these references into a render plan [1, 12]. These are software identities used to track preserved state, rather than measurements of image similarity.

Table 1. Production objects and their subsequent uses.

Object	Main content retained	Subsequent uses
Professional source project	Layers or objects, animation, materials, and compositing relationships	Direct editing, element extraction, and professional refinement
melo-gcs project	Gameplay, presentation, and element references	Reloading, continued variation, browser rendering, and on-demand native-project delivery
Element	Textures, scripts, effects, or components, with dependencies and usage information	Multimodal retrieval, selection, binding, and cross-project assembly
Finished video	Rendered images and their temporal sequence	Ad deployment, further compositing, or reconstruction reference

3.3 Delivery and Acceptance

The browser path delivers a project and a video, moving to native DCC refinement when its visual output cannot meet the design requirements. A task that directly edits an existing DCC project starts with the native project and may be completed in the original application.

Acceptance covers the requested edit, gameplay and preservation requirements, media integrity, and fitness for the task. Gameplay-rule compliance and visual structural integrity are basic requirements; overall acceptance also considers thematic expression, pacing, and suitability for ad placement. Rejected outputs return to the relevant editing step according to the issue.

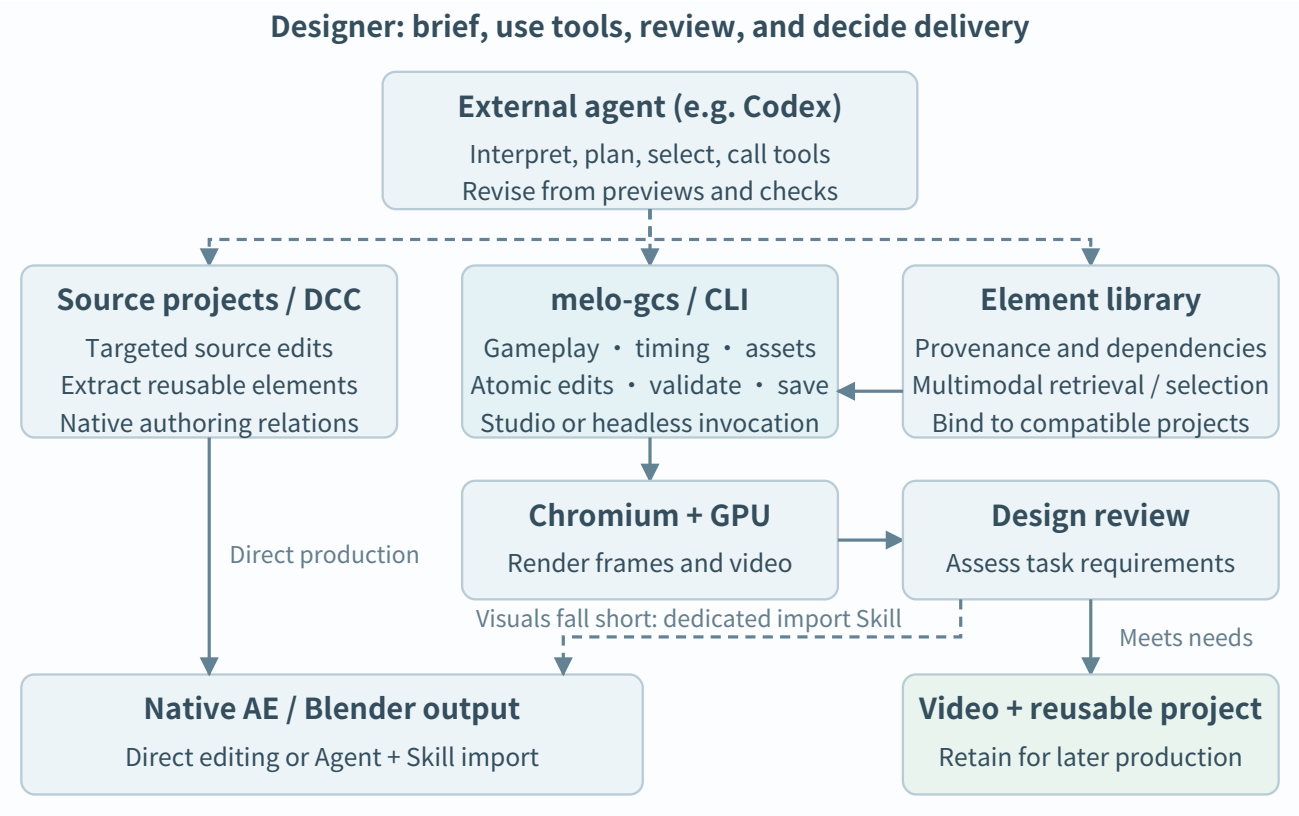
4 System Design and Division of Responsibilities

4.1 Overall Architecture

Figure 1 shows content sources, the external agent, melo-gcs, the element library, and delivery paths. Designers specify requirements and review results; they may also use melo-gcs Studio or professional applications directly. Codex hosts the external agent and its model-driven input interpretation, planning, asset selection, and tool calls. melo-gcs executes domain operations; replay and rendering of saved projects require no embedded language model [1].

DCC source projects can be edited directly or contribute reusable elements. The agent and melo-gcs CLI create candidate projects from reference videos. Existing melo-gcs projects can proceed directly to another editing round.

Reading schematic 1 · System roles and delivery paths



Redrawn from the final text. Not an original screenshot or a new experimental validation.

Original Figure 1 caption (original image not reproduced): Overall system architecture. Source projects can be edited directly or supply elements for multimodal retrieval and composition. melo-gcs performs headless rendering with Chromium and the GPU. Outputs meeting design requirements are delivered directly; otherwise, a dedicated import Skill transfers the project to After Effects or Blender for refinement. The figure shows only the role and position of professional import.

4.2 Source-Project Extraction, Element Storage, and Retrieval

The agent invokes professional tools to locate tile faces, materials, effects, or components and places suitable assets in the library with their provenance, dependencies, and usage information. Content unsuited to further decomposition remains as a complete project or component [3, 4]. Later tasks retrieve candidates from textual requests or visual references, select those compatible with the target project, and bind and assemble them.

The element library supports extraction, storage, retrieval, and use; melo-gcs runtime resource management resolves the resources and dependencies consumed by the current project [12]. We describe the interface between these responsibilities rather than the indexing, ranking, or multimodal retrieval algorithms.

4.3 Gameplay and Presentation Authoring in melo-gcs

melo-gcs treats gameplay, presentation, and rendering separately. Gameplay defines states, valid actions, and rule events; presentation schedules actions and feedback; rendering converts geometry, materials, lighting, and effects into images [1, 12]. Appearance edits can therefore reuse an existing gameplay recording.

The interfaces cover four game families: block placement, falling-block destruction, tray-based stacked-tile match-three, and Mahjong solitaire. Each retains its own rules and action definitions while sharing project production and tool invocation conventions.

Table 2. Main production capabilities and reuse patterns.

Capability	Main operations	Content retained or reused
Targeted source-project editing	Edit specified layers, objects, or effects in a copy	Unaffected animation and compositing relationships
Element retrieval and assembly	Select from text or visual references; resolve dependencies and bind	Registered assets, components, and usage information
Tile-face and material variants	Replace specified appearance and element references	Objects, gameplay, and events required by the task
Multilayer stack editing	Change layout, layering, and occlusion; check interactions	Applicable tiles, assets, and game rules
Pacing and action authoring	Adjust pauses, action intervals, feedback, or selections	Original trajectory or state recomputed from new actions
Reference-video reconstruction	Build a candidate project, render, compare, and revise	Saved melo-gcs project and associated elements
Batch and professional delivery	Render in the browser; deliver a DCC project when needed	Project files, videos, and on-demand native projects

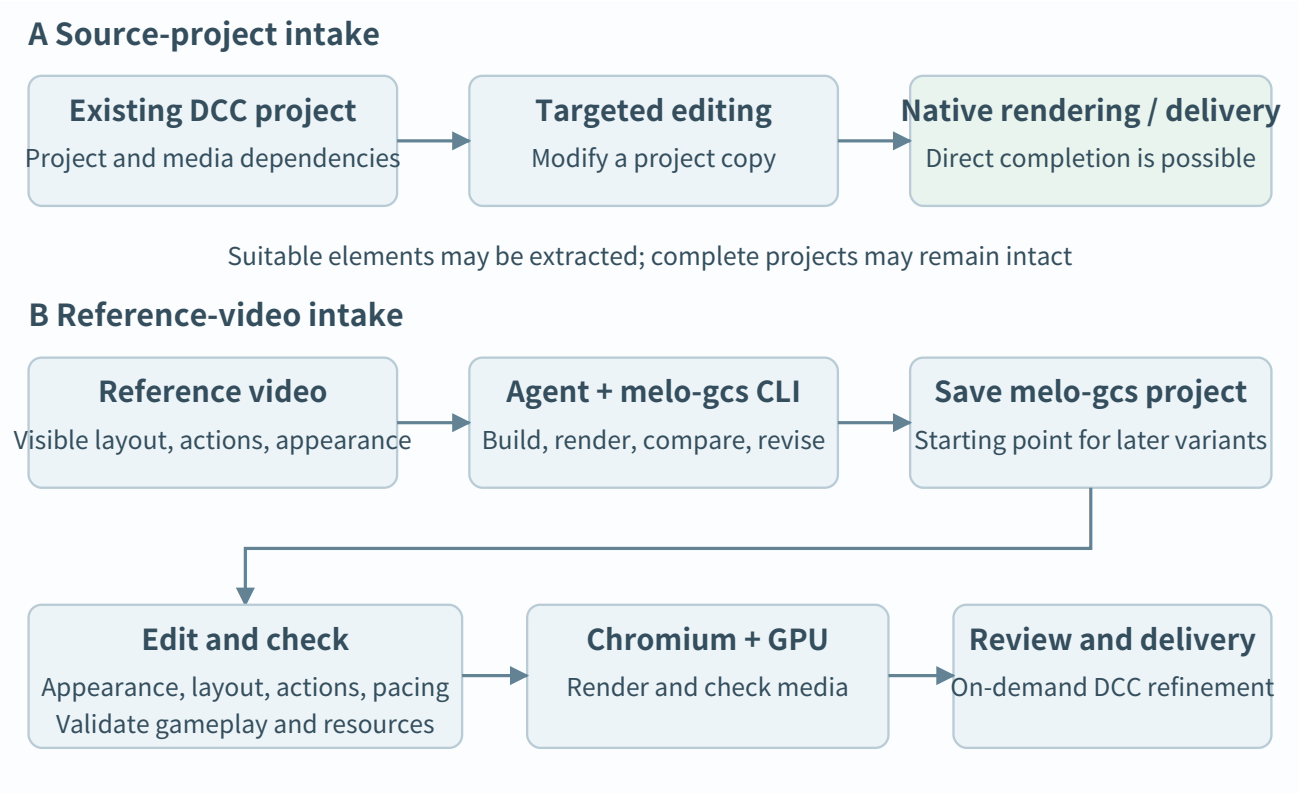
4.4 Atomic CLI Operations, Composable Skills, and Continued Editing

The CLI exposes atomic operations for configuration creation, appearance application, gameplay-recording validation, project generation, and rendering. Capability descriptions inform the agent about available tools. It reads structured results and selects the next operation; a Skill combines these calls, while melo-gcs performs the underlying work [1].

The maintainer-supplied melo-remix-looks Skill creates one validated Take and reuses it for each look [13]. In its falling-block workflow, skin apply changes appearance, take validate checks the unchanged Take against that configuration, document emit saves the project, and render optionally exports a video. A validation failure stops the variant. These operations also support fixed batch scripts; the external agent handles ad hoc planning and asset selection.

Figure 2 summarises intake, continued editing, and delivery.

Reading schematic 2 • Two intake paths and continued production



Redrawn from the final text. Not an original screenshot or a new experimental validation.

Original Figure 2 caption (original image not reproduced): Content intake and continued production. Source projects may be edited directly or organised into elements; reference videos are used to build melo-gcs projects. Variants pass through gameplay and resource checks, Chromium GPU rendering, media checks, and design review. Technical issues return to the relevant melo-gcs step. DCC refinement is used when the browser’ s visual result does not meet design requirements.

4.5 Reference-Video Reconstruction

After a reference video is supplied to Codex, the multimodal model uses its visual content and observed behaviour to invoke the melo-gcs CLI, create a project, arrange layouts and actions, and associate the required elements. melo-gcs renders the candidate project. The agent or designer compares it with the reference and revises it before saving a new executable project.

Recent internal reconstruction trials used a configuration recorded as GPT-6 Astra with extra-high reasoning effort. The author judged selected reconstructions to be visually close to their references. This is a qualitative observation for particular material and configurations: no aligned, per-frame reconstruction errors or systematic model comparison are reported.

After reconstruction, tile faces, materials, stacks, and pacing are edited through the same melo-gcs interfaces. The saved project and its element references become the starting point for subsequent production.

4.6 Interaction Pacing and Player-Like Behaviour

Pacing involves both timing and action selection. Hesitation, consecutive eliminations, and mistaken choices require different editing targets. Table 3 summarises the corresponding state handling.

Table 3. Pacing goals and state handling.

Presentation goal	Edits	State and trajectory handling
Hesitation	Add a pause before selection; adjust action intervals	Retain valid actions and reschedule their presentation
Fluent elimination	Shorten waits; coordinate actions and feedback	Retain valid actions and adjust pacing
Mistaken selection of a valid object	Select an allowed object that departs from the intended plan	Recompute tray, board, and subsequent eliminations
Click on an unavailable object	Stage an invalid attempt and feedback allowed by the rules	Do not force a valid move or elimination

These capabilities stage gameplay for advertisements. Action or layout changes are checked against the applicable rules, whereas timing-only changes retain the gameplay relationships specified by the task.

5 Browser Production and On-Demand DCC Refinement

5.1 Chromium GPU as a Production Rendering Path

melo-gcs uses hardware GPU rendering in Chromium, with Three.js providing scenes, materials, lighting, and effects for fixed or constrained viewpoints [9–11]. This path has been used in the team's production; batch and acceptance results are reported in Section 7.

The browser hardware-rendering path has been tested on NVIDIA GeForce RTX 3060 and RTX 4090 GPUs. All 72 videos in the batch described in Section 7 were exported by melo-gcs through Chromium GPU rendering. Hardware tests and batch timing were recorded separately.

Explicit frame indices drive rendering. The project determines when actions and feedback occur, so execution speed does not redefine their frame positions. Resource resolution, frame output, and video encoding are separate operations [1, 12]. Reusing frame-indexed state supports repeatable scheduling; pixel-identical output across browser versions, graphics drivers, or GPUs is not established by these timing contracts.

5.2 Headless Execution and Batch Production

In headless mode, an agent or script submits a project and output requirements. The production process resolves resources, performs checks, renders, and returns results. Producers need not open the workbench, manipulate a timeline, and start export for every video. They can still inspect browser previews or completed videos.

Each variant records changes to appearance, layout, or pacing and produces a video after checking. Execution results distinguish project generation, resource readiness, and completed video output. Local issues return to the appropriate step, while the project remains available for further editing.

For CLI tasks, execution summaries link input projects, editing operations, and output files. Screenshots of an interactive workbench are not needed to describe the execution process.

5.3 A Professional-Project Exit Driven by Design Requirements

Designers first review browser output. Material that meets the requirements is delivered directly; issues that can be addressed within melo-gcs lead to further project edits. When the browser's visual result cannot meet the design requirements, the melo-gcs project is exported and an external agent uses a dedicated import Skill to construct or modify a native After Effects or Blender project.

Designers can continue refinement and rendering in the native project. The professional interchange mappings and import Skill are internal implementations; this report describes the purpose of that path and its deliverables. The public melo-gcs project and replay contracts described in Section 3.2 are distinct from these private DCC integration details.

This exit supports further refinement of melo-gcs projects. Direct editing of an existing AEP or Blender source project can still be completed within its original application.

5.4 Implementation Configuration and Basic Checks

The public implementation described here is based on the 14 September 2026 software snapshot of melo-gcs 0.3.0-alpha.4. It records Three.js 0.185.1, Mediabunny 1.54.0, and TypeScript 5.8.3 as major dependencies, and Node.js 22.12 or later as a requirement [1]. Table 4 summarises software, hardware, and recorded model configurations. The relationship between this snapshot and the historical production batch is discussed in Section 8.3.

Table 4. Implementation and execution configuration summary.

Item	Recorded configuration	Use
Browser production	Chromium + GPU; headless support in melo-gcs	Preview, batch rendering, and video export
Tested GPUs	RTX 3060; RTX 4090	Browser hardware-rendering path tests
External agent host	Codex	Task interpretation, tool calls, and orchestration
72-video production batch	GPT/SOL (recorded configuration label)	18 melo-gcs browser-production groups, four outputs each
Recent video reconstruction	GPT-6 Astra; extra-high	Reference-video intake and subsequent variants
Professional applications	After Effects; Blender	Source-project reuse and on-demand native refinement

The preservation modes are frame-exact, semantic, and rule-only [12]. The first permits appearance changes while locking replay timing and frame rate; its name does not imply identical rendered pixels. The public compiler rejects director overrides, a changed frame rate, or a missing frame replay hash for a frame-exact recipe. The semantic mode retains gameplay actions while permitting presentation retiming; rule-only editing allows a new trajectory under the applicable rules. Gameplay validation and resource checks address declared constraints, while rendered media and creative suitability still require their own checks. Versioned capability and schema descriptions support subsequent editing.

The accompanying audio runtime and resource library support sound-effect retrieval or generation, with sounds scheduled against existing event timing [15, 16]. Audio is an auxiliary part of the production process.

6 Production Cases

6.1 Targeted Source-Project Editing and Element Reuse

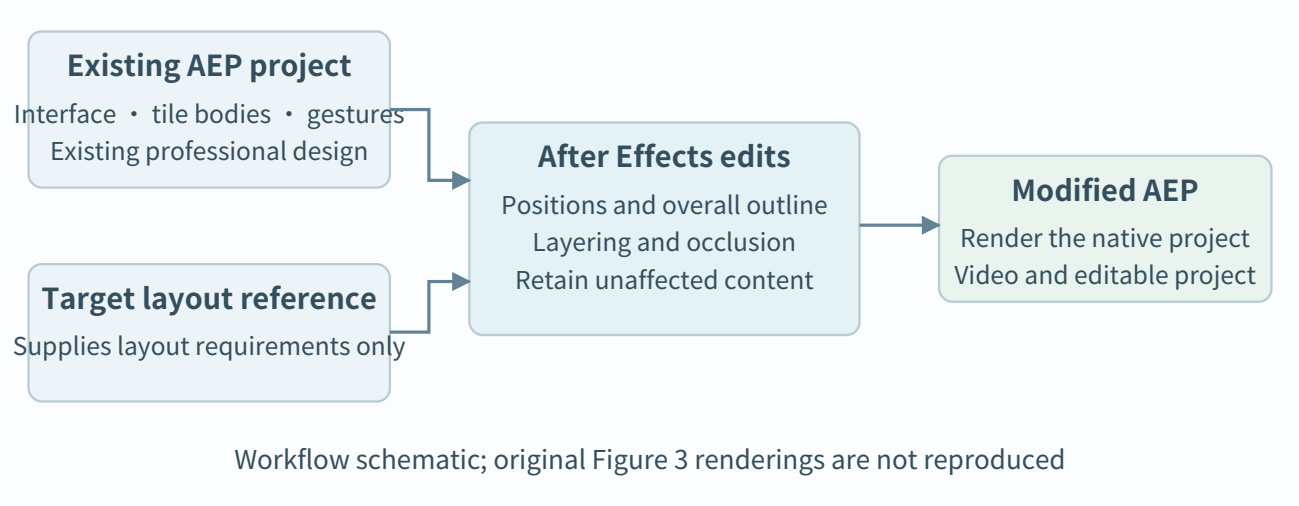
Existing workflows cover tile-face replacement, reference-guided layout rearrangement, effect replacement, and combinations of themes and structures [3, 4]. Tile-face and material tasks edit the layers or objects that consume the relevant assets. Layout tasks address position, layer order, occlusion, and interaction; effect tasks reuse applicable event positions and timing.

A source project can be edited as a whole or provide elements for other tasks. Tasks that can be completed by direct editing reuse the original project, while tasks sharing content can draw on the element library.

The repository records a combination of nine themes and four structures, and a historical collection combining 11 source projects with multiple replacement asset sets [3]. These records are kept separate from the 18 browser production task groups in Section 7.

Figure 3 illustrates reference-guided layout editing in an editable AEP. The original project and a layout reference are the inputs. Tile layers are located in After Effects and their positions, layering, occlusion, and overall outline are changed while retaining the original interface framework, tile-body appearance, and gesture assets. The reference specifies the target structure, and the result is rendered directly from the modified AEP.

Reading schematic 3 • Editing boundaries for source-project layouts



Redrawn from the final text. Not an original screenshot or a new experimental validation.

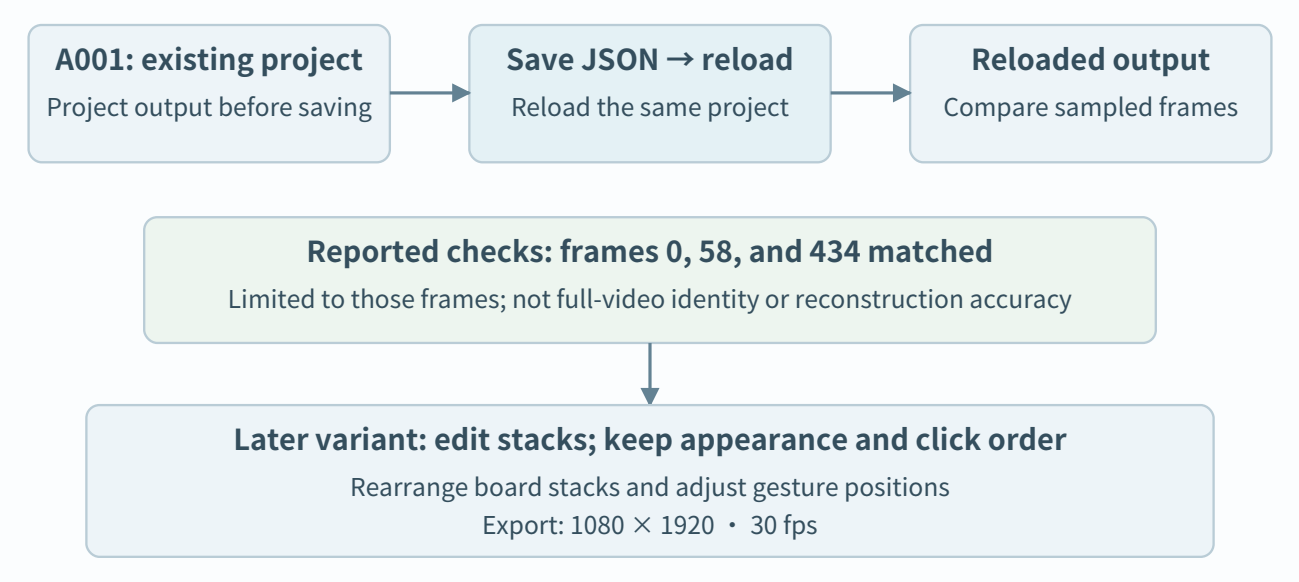
Original Figure 3 caption (original image not reproduced): Reference-guided layout rearrangement in an editable AEP. (a) A rendering of the original After Effects project. (b) The target stack reference. (c) The result after changing tile positions, layer order, and occlusion. The original interface framework, tile-body appearance, and gesture assets are retained; the reference supplies layout requirements only.

6.2 Continued Editing After Initial Intake

Figure 4 records saving, reloading, and stack editing for anonymised project A001. The case starts from an existing project, first checks save-and-reload consistency, and then produces a variant from the reloaded project.

After the project was saved as JSON and reloaded, pixel comparisons at frames 0, 58, and 434 matched the renderings before saving. The board stack was then rearranged and gesture positions adjusted while retaining tile faces, the background, and click order. The resulting variant was exported at 1080 × 1920 and 30 fps.

Reading schematic 4 • A001: save, reload, and continue editing



Redrawn from the final text. Not an original screenshot or a new experimental validation.

Original Figure 4 caption (original image not reproduced): Save-and-reload and continued editing for anonymised project A001. (a) Output before saving. (b) Output after reloading the saved project file. (c) Output after editing the stack in the reloaded project. All panels show frame 58 (30 fps, approximately 1.93 s). Pixel-consistency checks compare (a) and (b) at frames 0, 58, and 434.

The save-and-reload comparison concerns the same project, whereas the stack variant demonstrates subsequent editing. The three-frame check is limited to the sampled frames and does not establish full-video identity or reference-video reconstruction accuracy.

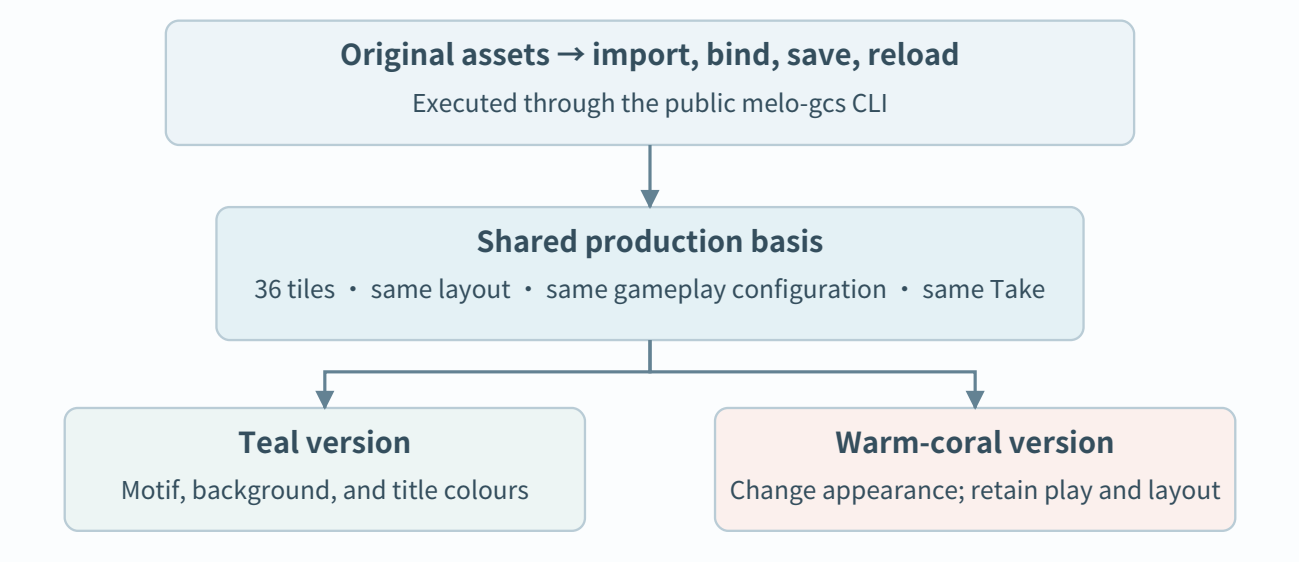
6.3 A Non-Commercial Example and the Online Workbench

Figure 5 uses transparent motifs and backgrounds created by scripts to demonstrate import, binding, saving, and variant production through the public melo-gcs CLI. Reference [14] provides the online workbench address; this case records CLI execution.

After import and binding, the project was saved as JSON, reloaded, and rendered. With 36 tiles, the board layout, gameplay configuration, and Take unchanged, edits to motif colours, the background, and title colour produced teal and warm-coral versions.

A001 varies stack geometry; this example varies appearance while preserving gameplay and layout. Both illustrate project reuse and are separate from the historical batch in Section 7.

Reading schematic 5 • Original-asset variants: preserved and changed content



Redrawn from the final text. Not an original screenshot or a new experimental validation.

Original Figure 5 caption (original image not reproduced): Import, saving, and variation with original assets. (a) Self-created transparent motifs and backgrounds. (b) A teal version rendered after CLI import, binding, saving, and reloading. (c) A warm-coral version made by changing motif colours, the background, and title colour in the saved project. Both versions retain the same 36 tiles, board layout, gameplay configuration, and Take.

7 Production Practice and Internal Acceptance

7.1 Browser Batch Output Across 18 Task Groups

One production batch comprised 18 task groups with four derived videos each, giving 72 outputs. Using a GPT/SOL-labelled configuration, melo-gcs rendered and exported the batch through Chromium GPU. The team recorded two hours for the production stage. Recent GPT-6 video-reconstruction practice is recorded separately.

The 18 groups follow the actual production grouping, each containing four derived outputs. This report does not enumerate the edit types for every group, and the group count does not denote either game categories or independently sampled scenes. The denominator of 72 is the set of exported outputs submitted to internal review; attempted candidates and intermediate retries are not separately tabulated here.

Table 5. Production batch and contextual records.

Item	Reported value	Measurement scope
Batch composition	18 groups × 4 variants = 72 videos	Multiple task groups and within-group derived outputs
Rendering path	melo-gcs / Chromium GPU	All 72 videos exported through the browser path
Production time	2 hours	Team-reported production-stage time
Model token charges	Below US$1 per output	Model-call charges for the batch; not total production cost
Manual After Effects / Blender production	1.5–2 videos per person-day	Team experience; tasks and working-day scope not standardised
Video-generation API + existing workflow	About 2 minutes per candidate	Screening and retries still required; not accepted-delivery time

Producing 72 outputs in two hours corresponds to 36 outputs per hour, or 100 seconds of batch time allocated per output. This is not single-request latency. The original records do not fully separate asset preparation, retries, and final review time, so no speed-up factor over manual editing or video-generation workflows is calculated.

7.2 Internal Acceptance Summary

According to the team's acceptance summary, two creative designers and one ad-optimisation specialist independently inspected the 72 outputs. Each video-level judgement was aggregated by agreement of at least two reviewers. Gameplay-rule compliance covers object identity, valid actions, and elimination order. Visual structural integrity covers the visible structure of tile faces, materials, and stacks. Overall production acceptance is the proportion approved for ad deployment, also considering thematic expression, pacing, and suitability for placement.

The same panel applied the same protocol to 72 outputs from the team's existing video-generation workflow. The workflow used a commercial video-generation API; product and provider names are withheld because of publication restrictions. Each of the three binary indicators uses all 72 reviewed videos in its respective workflow as the denominator. The indicators describe different dimensions and are not complements: a video may preserve the action sequence yet contain visible material or tile-face defects. Duration is reported separately as an output characteristic.

All 72 melo-gcs outputs passed the gameplay-rule compliance and visual structural integrity judgements. Fifty passed overall production acceptance, and all 50 were deployed. Four comparator outputs were approved; their deployment status was not fully recorded. The two 100% values are human judgements for this batch. Overall production acceptance remains 69.4%.

The comparator's 13 structurally intact videos are obtained by subtracting the 59 defect-bearing videos in the original record from 72. Neither group was evaluated against a common minimum-duration threshold. Approved comparator clips passed visual and elimination-logic review; results from workflows that extend clips by stitching are outside the table's scope.

Table 6. Descriptive internal acceptance summaries from author-confirmed records. The workflows were not evaluated under matched operating conditions.

Indicator	melo-gcs browser path (n = 72)	Video-generation API + existing workflow (n = 72)
Gameplay-rule compliance	72/72 (100%)	61/72 (84.7%)
Visual structural integrity	72/72 (100%)	13/72 (18.1%)
Video duration	All longer than 60 s	At most 30 s
Overall production acceptance	50/72 (69.4%)	4/72 (5.6%)

The groups shared reviewers, protocol, creative requirements, and output count, but the records do not establish paired cases. Operators, iteration budgets, and retry conditions were not strictly matched, and the comparator API version and access date were not fixed. Output durations also differed, affecting the scope of temporal consistency being inspected. Reviewer blinding and randomised presentation are not reported. Table 6 describes these two production workflows under their recorded conditions; it does not estimate a causal effect or a general performance difference between model families.

7.3 Charges and Accepted Output

Recorded model token charges were below US$1 per produced video for the same batch. Under that accounting scope, the recorded batch charge is below US$72. Allocating it across the 50 approved videos gives a batch-average token charge below US$1.44 per approved video. This is a derived upper bound on the allocated average, not a measured mean or a bound on each individual approved video's cost. The report does not provide a stage-level breakdown of model calls or retry charges.

Dividing the 50 approved outputs by the reported two-hour production stage gives 25 approved outputs per production-stage hour. Review time is not separately itemised, so this quantity is not an end-to-end accepted-delivery rate. Asset organisation, system development, hardware, professional software, and human review were not costed separately.

The batch records browser-based production across multiple variant groups, including outputs accepted for deployment. The available data do not isolate the benefits of asset reuse, fixed scripts, agent decisions, concurrency, or caching.

8 Design Experience and Scope

8.1 Retain Design Work Before Expanding Reuse

Source projects preserve directly editable authoring relationships. Supporting complete-project editing first, then extracting elements from recurring requirements, ties asset organisation to concrete tasks. Multimodal retrieval supports selection; the melo-gcs project records how selected elements are used in the current production.

Complete projects and local elements serve different reuse granularities. Initial preparation and later derivation should be accounted for separately. Extensive preparation may not be worthwhile for one-off or rarely repeated tasks.

8.2 Browser-First Production and Task-Driven Professional Refinement

For the fixed or constrained viewpoints covered here, Chromium GPU rendering supplies production deliverables and headless mode supports agent and script orchestration. Professional applications supply existing assets and provide refinement when the browser's visual result is insufficient. Existing DCC projects can also be produced directly in their native applications.

Models interpret inputs, select assets, and plan. Projects and domain tools retain production results and execute subsequent edits. Model upgrades may expand reference-video reconstruction, while saved projects continue to be edited through established interfaces.

8.3 Scope of the Production and Quality Evidence

The 72-video batch was exported by melo-gcs through Chromium GPU. RTX 3060 and RTX 4090 tests establish operation of the hardware-rendering path. The historical batch lacks complete concurrency, stage-level timing, and version records corresponding to the current implementation, preventing GPU performance comparisons or estimates of version-specific benefits. Variants within the 18 groups share resources; aggregate proportions are not treated as estimates from 72 independent tasks.

Internal acceptance follows the team's deployment criteria. Individual judgements and inter-rater agreement are not reported, limiting analysis of reviewer disagreement and panel composition. Aggregate counts also cannot separate the contributions of structure, style, and other requirements to rejection; in particular, the reasons for the 22 rejected melo-gcs outputs are not itemised. The three-frame comparison in Section 6.2 checks project save-and-reload. Reference-video reconstruction is a separate qualitative observation without measured errors under standardised temporal alignment and image-size conditions.

The cases and production records support browser use for the stated tasks, not all camera motion, plugins, or game behaviours. Gameplay and structural checks remain complementary to designers' assessments of creative value and task suitability.

8.4 Disclosure Scope and Further Records

This report describes the element library's extraction, organisation, retrieval, and assembly functions without evaluating its retrieval algorithms. Professional import is an internal implementation that cannot be reproduced from the public documentation alone. Lossless round-tripping between arbitrary professional projects and melo-gcs has not been established. Pacing controls stage gameplay rather than model empirical player behaviour; click-through rate, conversion, advertising return, and audio quality are outside the evaluation scope.

Section 6 presents source-project editing, save-and-reload, and original-asset variation cases. Subsequent comparisons can use these tasks with fixed inputs, budgets, and evaluation criteria to measure reconstruction error, the effort required for later edits, and acceptance separately.

9 Conclusion

This report presents a reusable production system for casual puzzle-game advertisements that brings design work distributed across professional source projects into subsequent production. Complete projects can be edited directly, while suitable content can be extracted into an element library for multimodal retrieval and assembly. melo-gcs projects retain gameplay recordings, presentation timing, and element references; atomic CLI operations support appearance, layout, action, and pacing edits. Reference video provides another project-intake path when source projects are unavailable.

melo-gcs exports videos through Chromium GPU and headless execution, moving to native After Effects or Blender refinement when browser output cannot meet design requirements. Hardware rendering has been tested on RTX 3060 and RTX 4090. The reported production batch comprised 18 groups of four variants: 72 videos were exported during a two-hour production stage, and 50 (69.4%) passed internal acceptance and were deployed. Recorded model token charges were below US$1 per produced video under the accounting scope described in Section 7.3.

The source-project layout-editing, A001 save-and-reload, and original-asset variation cases show how existing designs and saved projects support continued editing, while the production record describes browser-based batch output. Combining existing design resources, editable projects, domain tools, and on-demand professional refinement provides a practical production route alongside direct video generation.

Materials Availability and Disclosure

melo-gcs, its complementary professional workflows, and the online workbench are listed in References [1, 3, 4, 13, 14]. The software and professional-editing references link to the cited versions, with the implementation described in Section 5.4. Public project and replay contracts are available in that source. Commercial source projects, private professional interchange mappings, and DCC import Skills are not distributed with this report. Section 6 uses de-identified case images

and original non-commercial assets. The historical production, cost, and review records are retained separately by the authors; the submission includes their aggregate results, not the underlying per-video records.

AI tools assisted with literature organisation and manuscript preparation; the authors are responsible for the report's content.

References

[1] dongyuan21. Melo Game Creative Studio: Repository and Architecture Documentation. Software repository and project documentation, 2026. Accessed 20 September 2026.

[2] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. VACE: All-in-One Video Creation and Editing. arXiv:2503.07598, 2025. DOI: https://doi.org/10.48550/arXiv.2503.07598.

[3] dongyuan21. AE–Blender Skill: Creative Workflows and Showcase. Project documentation, 2026. Accessed 20 September 2026.

[4] dongyuan21. Reference-Guided Stack Layout Editing in After Effects. Project documentation, 2026. Accessed 20 September 2026.

[5] Ian Huang, Guandao Yang, and Leonidas Guibas. BlenderAlchemy: Editing 3D Graphics with Vision-Language Models. arXiv:2404.17672, 2024. DOI: https://doi.org/10.48550/arXiv.2404.17672.

[6] Yunqi Gu, Ian Huang, Jihyeon Je, Guandao Yang, and Leonidas Guibas. BlenderGym: Benchmarking Foundational Model Systems for Graphics Editing. arXiv:2504.01786, 2025. DOI: https://doi.org/10.48550/arXiv.2504.01786.

[7] Zhenran Xu, Longyue Wang, Jifang Wang, Zhouyi Li, Senbao Shi, Xue Yang, Yiyu Wang, Baotian Hu, Jun Yu, and Min Zhang. FilmAgent: A Multi-Agent Framework for End-to-End Film Automation in Virtual 3D Spaces. arXiv: 2501.12909, 2025. DOI: https://doi.org/10.48550/arXiv.2501.12909.

[8] Mathias Bynens and Peter Kvitek. Chrome Headless Mode. Chrome for Developers. Accessed 14 September 2026.

[9] Chromium contributors. Using GPU Hardware in Headless Chrome. Chromium documentation. Accessed 14 September 2026.

[10] Three.js contributors. WebGLRenderer. Three.js documentation. Accessed 14 September 2026.

[11] Three.js contributors. MeshStandardMaterial. Three.js documentation. Accessed 14 September 2026.

[12] dongyuan21. Melo Game Creative Studio: Headless Core v1. Project architecture documentation, 2026. Accessed 20 September 2026.

[13] dongyuan21. melo-gcs: One Take, Multiple Looks. Workflow Skill supplied by the melo-gcs maintainers, 2026. Accessed 20 September 2026.

[14] dongyuan21. Melo Game Creative Studio Online Workbench. Workbench address supplied by the author, 2026.

[15] dongyuan21. nanoAuralRuntime. Software repository, 2026.

[16] dongyuan21. Game Audio Asset Library. Project repository, 2026.